

Ensemble v7.0.0: Configuration-Driven Tools Plan

Configuration-Driven Tools for Ensemble Plugins — v7.0.0 Plan

Baseline: `Sunstone-Partners/ensemble main @ 4967777` (root version **6.9.2**) **Author:** Mike Devenney · 2026-08-11 (v3 — `tracker` axis with `azure-boards` committed to 7.1.0) **Status:** Proposal — not yet started

Scope: two configuration layers

Ensemble hardcodes two different kinds of “tools,” and they need different mechanisms:

Layer	Question it answers	Resolved	Example
1. Tool grants	Which harness tools is an agent <i>allowed</i> to call?	Build time (artifact generation)	<code>Read, Write, Bash</code> in agent frontmatter
2. Toolchain providers	Which development tools does an agent <i>drive</i> when working?	Run time (when the agent executes in a project)	<code>gh</code> vs <code>glab</code> vs <code>az repos</code> ; <code>git-town</code> vs vanilla <code>git</code> ; <code>Claude Code</code> vs <code>Pi</code> vs <code>Copilot CLI</code>

Layer 1 is a generation concern — plugins ship static markdown, so grants must be baked in when artifacts are generated. Layer 2 is a project concern — the same installed plugin should file a PR on GitHub in one repo and a merge request on GitLab in another, so providers must resolve where the agent runs.

Layer 1 — Tool grants registry

The problem

Tool grants are hardcoded in five places, in three representations, with nothing keeping them in agreement:

#	Surface	Where	What's hardcoded
1	Agent sources	33 files, <code>packages/*/agents/*.yaml</code>	<code>metadata.tools</code> — required array of free-form strings
2	Generated agents	<code>packages/*/agents/*.md</code>	<code>tools: [...]</code> frontmatter, copied verbatim by <code>scripts/lib/agent-transformer.js</code>
3	Command sources	6 files with <code>metadata.allowed_tools</code>	comma-joined into <code>allowed-tools:</code> by <code>scripts/lib/command-transformer.js</code>
4	Schemas	<code>schemas/agent-yaml-schema.json</code> , <code>command-yaml-schema.json</code>	tools are unvalidated strings — a typo like <code>Grpe</code> ships silently
5	Runtime translators	OpenCode: <code>TOOL_PERMISSION_MAP</code> in <code>scripts/generate-opencode/src/agent-translator.js</code> ; Pi: <code>CLAUDE_CODE_ONLY_TOOLS</code> in <code>packages/pi/src/transformers/agent-transformer.ts</code> ; Codex: emits no tools	Per-runtime semantics live as code constants; unknown tools are silently dropped

The tool vocabulary actually in use: `Read`, `Write`, `Edit`, `Bash`, `Grep`, `Glob`, `Task`, `TodoWrite`, `Skill`, `AskUserQuestion` .

The design

One registry file, `config/tools.yaml` — canonical vocabulary, per-runtime semantics, and named composable toolsets. Initial values lifted 1:1 from the two hardcoded tables, so it is behavior-preserving by construction:

```

version: 1
tools:
  Read: { opencode: { read: allow }, pi: keep }
  Grep: { opencode: { read: allow }, pi: strip }
  Glob: { opencode: { read: allow }, pi: strip }
  Write: { opencode: { edit: allow }, pi: keep }
  Edit: { opencode: { edit: allow }, pi: keep }
  Bash: { opencode: { bash: ask }, pi: keep }
  Task: { opencode: null, pi: strip } # Claude Code only
  TodoWrite: { opencode: null, pi: strip }
  Skill: { opencode: null, pi: strip }
  AskUserQuestion: { opencode: null, pi: strip }

toolsets:
  read-only: [Read, Grep, Glob]
  developer: [Read, Write, Edit, Bash, Grep, Glob]
  orchestrator: [developer, Task, TodoWrite, AskUserQuestion] # toolsets compose

```

Each **runtime is a column** in this registry. That makes “Editor/CLI” a first-class, extensible axis: adding a runtime (e.g. Copilot CLI) means adding a column plus a generator under `scripts/generate-
<runtime>/` — no changes to agent YAMLS.

Loader, `scripts/lib/tool-registry.js`, with override resolution (first hit wins, deep-merged over the repo default):

1. `--tools-config <path>` CLI flag on the generators
2. `ENSEMBLE_TOOLS_CONFIG` env var
3. `$XDG_CONFIG_HOME/ensemble/tools.yaml` — reusing `packages/core/lib/config-path.js`
4. Repo default `config/tools.yaml`

Authoring change (the breaking part): `metadata.tools` entries may be tool names or toolset names, expanded at generate time; every entry must resolve against the registry — unknown names become a hard validation error in v7 (today: silently accepted). Generated `.md` keeps the exact same concrete-list shape, so runtimes never see a toolset name.

Consumers: `generate-markdown.js` (expand + validate), OpenCode translator (delete `TOOL_PERMISSION_MAP`), Pi transformer (delete `CLAUDE_CODE_ONLY_TOOLS`), `validate-all.js` (registry schema check + every reference resolves).

Layer 2 — Toolchain provider configuration

The problem

Agents don't just hold tool grants — their prose drives specific development tools, and those choices are welded into content across the plugin set (all counts from `main`):

Assumption	Hardcoded in	Examples
GitHub + <code>gh</code> CLI	17 YAML sources mention GitHub; 11 files invoke <code>gh pr/issue/api/run</code>	<code>fix-issue</code> , <code>beads-build</code> , <code>implement-bead</code> , <code>release</code> , <code>create-prd</code> , <code>code-reviewer</code> , <code>deep-debugger</code> , <code>github-specialist</code>
git-town	7 YAML sources	<code>ensemble-orchestrator</code> , <code>git-workflow</code> , <code>implement-trd</code> , <code>implement-trd-beads</code> , <code>refine-trd</code> , <code>beads-build</code> , <code>implement-bead</code>
Beads (<code>br</code> / <code>bv</code>)	16 YAML sources	the whole <code>implement-trd-beads</code> pipeline
Runtime = Claude Code	canonical source format	OpenCode/Codex/Pi are downstream translations

A team on GitLab or Azure DevOps gets agents that speak the wrong CLI, the wrong terminology (PR vs MR vs pull request into Azure Repos), and the wrong SDLC linkage (issues vs work items). Today their only option is forking plugin content.

The design: capability axes + provider skills + detection

Ensemble already has the exact pattern this needs, shipping today:

- **Provider-style skills:** `packages/git/skills/git-town/`, `packages/infrastructure/skills/managing-jira-issues/`, `managing-linear-issues/` — self-contained “how to drive this tool” documents.
- **Detection + dynamic skill loading:** `packages/core/skills/framework-detector/` and `test-detector/` (with `packages/core/lib/detect-framework.js` + patterns JSON); `backend-developer` already loads NestJS/Rails/Phoenix/.NET skills based on detection signals.

Layer 2 generalizes that pattern from *frameworks* to the *toolchain*:

Capability axes, declared in `config/toolchain.yaml`:

```

version: 1
axes:
  runtime: # Editor/CLI – build-time axis (see Layer 1); listed here for install tooling
    providers: [claude-code, opencode, codex, pi]
    default: claude-code
  scm: # source control + SDLC platform
    providers: [github, gitlab, azure-devops]
    default: github
    detect:
      - { signal: "git remote url matches github.com", provider: github }
      - { signal: "git remote url matches gitlab", provider: gitlab }
      - { signal: "git remote url matches dev.azure.com", provider: azure-devops }
  gitflow: # branching workflow tool
    providers: [git-town, vanilla]
    default: git-town
    detect:
      - { signal: "git town config present", provider: git-town }
  tracker: # work-item system of record (execution layer stays Beads – see below)
    providers: [beads, github-issues, azure-boards, jira, linear]
    default: beads

```

Resolution order (first hit wins): project file `.ensemble/toolchain.yaml` in the consuming repo → user XDG `$XDG_CONFIG_HOME/ensemble/toolchain.yaml` → detection → repo defaults. Defaults equal today's hardcoded behavior, so an unconfigured project behaves exactly as v6 does.

Provider skills — one skill per (axis, provider), each documenting the CLI, terminology, and workflow equivalents:

<code>packages/git/skills/scm-github/SKILL.md</code>	# extracted from today's gh-CLI content
<code>packages/git/skills/scm-gitlab/SKILL.md</code>	# glab; "merge request" vocabulary
<code>packages/git/skills/scm-azure-devops/SKILL.md</code>	# az repos / az boards; work items, #NNNNN linking
<code>packages/git/skills/gitflow-git-town/SKILL.md</code>	# today's git-town skill, re-homed under the axis
<code>packages/git/skills/gitflow-vanilla/SKILL.md</code>	# plain branch/rebase/push flow

Each provider skill carries a small **vocabulary block** (change-request noun, work-item noun, CLI binary, link syntax) so shared agent prose can stay neutral.

Toolchain detector — `packages/core/skills/toolchain-detector/` mirroring `framework-detector`: reads git remote URL, git-town config, `.beads/`, and reports resolved providers per axis. Agents consult it once at task start.

Content refactor — the sweep that makes it real: the ~20 YAML sources that hardcode `gh` / `git-town` / `GitHub` prose are rewritten capability-neutral ("create a change request using the resolved `scm` provider — load its skill"), exactly how `backend-developer` defers framework specifics to framework skills today. `github-specialist` becomes the thin GitHub provider specialist behind the `scm` axis (its content seeds `scm-github`).

Enforcement — `scripts/lint-provider-refs.js` (modeled on the existing `scripts/lint-model-ids.js`): CI fails if shared agent/command sources reference a provider binary (`gh` , `glab` , `git town` , `az repos`) outside provider skills. This is what stops the hardcoding from creeping back.

What stays put: Beads remains the *execution* layer for `implement-trd-beads` (dependency graph, ready-work queue). The `tracker` axis governs the *system of record* the workflow links to and reports into — mirroring the “Beads executes, the tracker records” split that ensemble-based teams already run.

Why this is one major version

Per `docs/VERSIONING_STRATEGY.md` , breaking changes require beta/rc cycles, a migration guide, and explicit opt-in:

1. **Validation tightens** (Layer 1): unknown tool names go from silently accepted to hard failure — downstream forks with custom agent YAMLs can break at `npm run validate / generate` .
2. **Translator contracts change** (Layer 1): `TOOL_PERMISSION_MAP` and `CLAUDE_CODE_ONLY_TOOLS` are deleted.
3. **Content contracts change** (Layer 2): ~20 agents/commands are rewritten provider-neutral; anyone patching or extending that prose downstream must re-base.

Installed users are protected twice over: Layer 1 regeneration is byte-identical (golden test), and Layer 2 defaults resolve to today’s exact toolchain (GitHub + `gh` , `git-town`, Beads). Upgrading to 7.0.0 without adding any config is behaviorally inert.

The payoff after 7.0.0: adding a provider (GitLab, Azure DevOps) or a runtime (Copilot CLI) is a **minor** release — a new skill or a new registry column + generator — never another major.

Rollout — two workstreams, enforcement flips only at 7.0.0

Workstreams A and B are independent until the 7.0 beta train; they can run in parallel. Maps onto the repo’s TRD `### PR N:` boundary format if run through `/ensemble:create-trd` .

Workstream A — tool grants registry

PR	Version	Shippable state
A1 Registry + schema + loader, warn-only	6.x minor	<code>npm run validate</code> warns on unregistered tool names; nothing else changes
A2 Generators consume registry; toolset expansion; golden byte- identical test over all 33 agents + 6 commands	6.x minor	Authors may use toolsets; existing YAMLS generate identically
A3 OpenCode + Pi translators read registry; hardcoded tables deleted; snapshot tests	6.x minor	Runtime artifacts registry-driven, output unchanged

Workstream B — toolchain providers

PR	Version	Shippable state
B1 config/toolchain.yaml + schema + loader (project → XDG → detect → default) + toolchain- detector skill	6.x minor	Detection reports resolved providers; nothing consumes it yet
B2 Extract current GitHub/git-town content into scm-github and gitflow-git-town provider skills (content move, no behavior change); provider-skill template + vocabulary block format	6.x minor	Provider matrix exists with today's two providers
B3 Content refactor: ~20 YAML sources rewritten provider-neutral, resolving via toolchain config; tracker touchpoints abstracted — work-item linking in commit messages and change-request bodies, status reporting on task completion — behind the tracker axis with beads as sole provider; add lint-provider- refs.js warn-only	7.0.0-beta	Agents drive whatever the resolved provider is; defaults preserve v6 behavior; tracker axis is provider-ready

Release train

PR	Version	Shippable state
R1 Migrate all agent/command YAMLS to toolsets; flip Layer 1 validation and provider-ref lint warn → error; migration guide in <code>docs/guides/</code> ; docs updates (<code>CLAUDE.md</code> , <code>VERSIONING_STRATEGY.md</code> note)	7.0.0-beta.1	Full contract enforced; guide published
R2 Beta soak per versioning strategy → stable; release notes via <code>scripts/generate-release-notes.js</code> ; version wave: root / <code>packages/full</code> / <code>marketplace.json</code> in lockstep (<code>validate-version-sync.js</code>); major-bump every plugin whose YAML or translator contract changed (<code>packages/pi</code> 1.5.0 → 2.0.0, <code>packages/opencode</code> , YAML-carrying packages)	7.0.0	Tagged, in the marketplace; 6.x users upgrade opt-in
R3 Tracker fast follow: <code>tracker-azure-boards</code> provider skill — <code>az boards</code> CLI, work-item linking syntax (<code>#NNNNN</code> in commits, <code>AB#NNNNN</code> where GitHub-ADO linking is in play), state transitions on task completion; end-to-end test against a real ADO project	7.1.0	Teams can point the system of record at Azure Boards while Beads keeps executing; provider-skill-only change, no content sweep
R4+ <code>scm-gitlab</code> , <code>scm-azure-devops</code> , <code>gitflow-</code>	7.x minors	Provider matrix grows without majors

PR	Version	Shippable state
vanilla provider skills; further tracker providers (github- issues , jira , linear — the existing jira/linear skills seed these); candidate Copilot CLI runtime generator		

Testing

- **Golden regeneration test** (load-bearing, Layer 1): registry-driven generation of every shipped YAML $\equiv$ v6.9.2 committed output.
- **Default-toolchain equivalence** (load-bearing, Layer 2): with no config and GitHub detection, refactored commands produce the same instructions/skill loads as v6 content (snapshot per command).
- Registry + toolchain loader units: precedence, deep-merge, missing/invalid file, toolset cycle detection, detection-signal parsing.
- Provider skill contract test: every (axis, provider) skill carries the required vocabulary block; every axis has a skill for every declared provider.
- `lint-provider-refs.js` self-test + updated `packages/opencode` / `packages/pi` snapshots.

Explicitly out of scope

- **Runtime enforcement of grants** — the permitter hook governs Bash permission expansion; orthogonal.
- **Codemod for third-party YAMLs** — the migration is mechanical; the guide covers it.
- **Migrating Beads off the execution layer** — `tracker` axis covers the system of record only; replacing the execution engine is its own project.
- **Copilot CLI generator in 7.0** — the architecture makes it a 7.x minor; scoping it into the major would delay everything else.
- **Hand-authored .md commands** (`packages/router/commands/*.md`) — don't pass through the generator; unchanged.

Open questions for Sunstone review

1. **Toolset names/granularity** (Layer 1) — proposed `read-only` / `developer` / `orchestrator` .

2. **Axis set for 7.0** — *decided 2026-08-11*: `runtime`, `scm`, `gitflow` ship fully in 7.0; `tracker` ships its config surface + `beads` provider in 7.0, and **azure-boards is committed as the 7.1.0 fast follow** (R3). Still open: timing for `github-issues` / `jira` / `linear` tracker providers.
3. **Where provider skills live** — proposed `packages/git/skills/` for `scm/gitflow` (they're git-adjacent); alternative is a new `packages/toolchain` plugin. Affects marketplace packaging.
4. **Project-local overrides** — `.ensemble/toolchain.yaml` is essential for Layer 2 (per-repo SCM); should Layer 1 grants also honor a project-local file, or stay XDG-only?
5. **Pi mapping richness** — Pi allows `Read`, `Write`, `Edit`, `Bash`, `ask_user`; is `keep` / `strip` sufficient long-term?

How this lands (fork process)

Each PR: branch from `main` (verified mirror of Sunstone `main`), PR against `Sunstone-Partners/ensemble main`, and separately merge into this fork's `dev` for local dogfooding. The 7.1 `azure-boards` tracker is the piece this fork's org needs first — "Beads executes, ADO records" is already the org's working pattern, so it's the natural dogfooding candidate on `dev` while upstream review proceeds. `scm-azure-devops` can pull forward from R4 if ADO-hosted repos need the `scm` axis sooner.